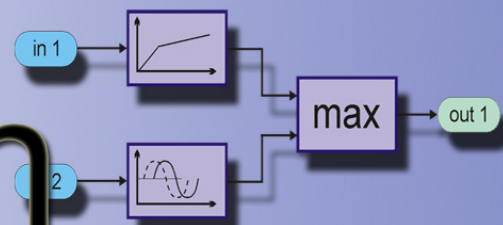


PExSim

Badanie i Symulacja Procesu



Nazwa dokumentu:

"C/C++ scripts" - biblioteka modułu PExSim Opis funkcjonalności

Wersja soft.: 0.3.0 beta

Wersja doc.: 0.3.0-1 beta

Finansowanie: Projekt zamawiany PW-004/ITE/03/2005 „Opracowanie i zastosowanie w pilotowym procesie pakietu programowego służącego do badania dynamiki procesu, reakcji zainstalowanych układów sterowania oraz symulacji działania w celu kształcenia obsługi – *Process Discoverer*”



Politechnika Warszawska
Instytut Automatyki i Robotyki



Spis treści

1.	Wstęp	2
2.	Zasady tworzenia skryptów	2
2.1.	Składnia – wzorzec skryptu	2
2.2.	Stan obiektu i zmienne globalne użytkownika	3
2.3.	Etapy tworzenia i uruchamiania skryptu	5
2.4.	Katalog skryptów, kompilacja	5
3.	Opis bloków funkcyjnych	7
3.1.	Script C/C++	7
3.1.1.	Okno: wybór i zarządzanie skryptami	7
3.1.2.	Opcje konfiguracji środowiska edycji skryptów	9

Wykaz załączników

<u>Dodatek A.</u>	Kod źródłowy wzorca skryptów C/C++ „script_template.cpp”
-------------------	--



1. Wstęp

Biblioteka C/C++ SCRIPTS dostarcza bloki funkcyjne umożliwiające uruchamianie własnych procedur obliczeniowych napisanych w postaci skryptów języka C/C++. Skrypty takie kompilowane są do postaci dynamicznie ładowanych bibliotek, które to wykorzystywane są przez moduł PEXSIM w bieżących obliczeniach.

2. Zasady tworzenia skryptów

2.1. Składnia – wzorzec skryptu

Przy pisaniu skryptów można wykorzystywać praktycznie dowolne konstrukcje i mechanizmy języka C i C++ (tworzenie własnych typów i metod, tworzenie zmiennych dynamicznych, łączenie zewnętrznych bibliotek, itd.) jakkolwiek struktura tworzonego skryptu musi odpowiadać zdefiniowanemu wzorcowi.

Skrypty użytkownika tworzone są na podstawie wzorca „script_template”. Każdy skrypt składa się z pięciu metod, które muszą lub mogą być uzupełniane przez użytkownika. W ciałach tych metod użytkownik umieszcza kod realizujący zaplanowany przez niego algorytm. Wyróżniamy następujące metody:

- csInit

Metoda ta jest wywoływana zawsze gdy skrypt jest przypisywany do bloku funkcyjnego w środowisku symulacyjnym, także podczas wczytywania z pliku konfiguracji modułu PEXSIM. W metodzie tej trzeba podać liczbę wejść i wyjść „algorytmu” – czyli bloku funkcyjnego, do którego przypisany będzie skrypt:

```
_input_count = 3;
_output_count = 2;
```

W metodzie tej można przykładowo utworzyć zmienne dynamiczne użytkownika.
- csRelease

Metoda ta jest wywoływana zawsze gdy skrypt zostaje odłączony od danego bloku, blok z danym skryptem jest usuwany albo moduł PEXSIM kończy działanie. ([aktualnie niedostępna](#))

W metodzie tej można przykładowo usunąć zmienne dynamiczne użytkownika.
- csBeforeSimStart

Metoda wywoływana przed rozpoczęciem symulacji modułu PEXSIM (! nie w każdym kroku). Metoda nie musi być zaimplementowana.

Metoda ta może być wykorzystana przykładowo do jednorazowego przeliczenia parametrów zależnych od aktualnych opcji symulacji (np. okresu próbkowania).
- csAfterSimStop

Metoda wywoływana po zakończeniu symulacji modułu PEXSIM (! nie w każdym kroku). Metoda nie musi być zaimplementowana.

Metoda ta jest rzadko wykorzystywana.

- `csBeforeCalc` Metoda wywoływana przed rozpoczęciem każdego kroku symulacji modułu PExSim. Metoda nie musi być zaimplementowana. Uwaga: aktualne wartości wejść są nieprawidłowe!
Metoda ta jest rzadko wykorzystywana.
- `csAfterCalc` Metoda wywoływana po zakończeniu każdego kroku symulacji modułu PExSim. Metoda nie musi być zaimplementowana. Uwaga: aktualne wartości wyjść nie są ostateczne!
Metoda ta może być przykładowo wykorzystania do zapamiętania / aktualizacji stanu obiektu.
- `csCalculate` Metoda wywoływana w każdym kroku symulacji. W niej obliczane są oraz ustawiane wartości i statusy wyjść (jeśli takowe istnieją).

```

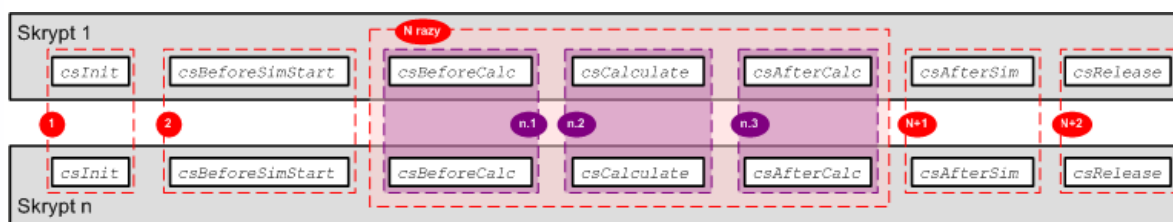
_out_val[i] = 23.5;
_out_status[i] = 0;

```

Uwaga! - Metoda nie będzie wywoływana jeśli do zdefiniowanych wejść bloku nie będą podpięte sygnały (wyjścia innych bloków) lub gdy metoda ta nie będzie wywoływana dla bloków „wcześniejszych”.

Uwaga! – Możliwa jest sytuacja, w której metoda ta zostanie wywołana kilka razy w jednym kroku symulacji (pomiędzy `csBeforeCalc` i `csAfterCalc`).

Dokładny opis parametrów funkcji znajduje się we wzorcu skryptu, którego listing (plik: `script_template.cpp`) zamieszczono w dodatku A. Poniżej pokazano diagram kolejności wywołania poszczególnych metod.



Rys. 1). Kolejność wywołania metod wielu skryptów podczas symulacji.

2.2. Stan obiektu i zmienne globalne użytkownika

Pamiętanie stanu obiektu

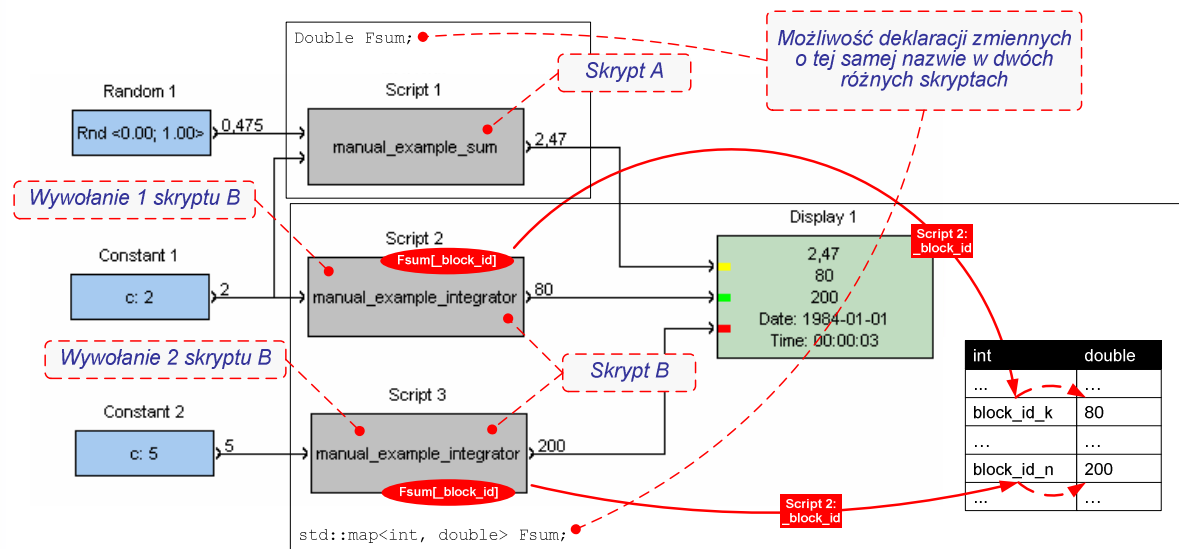
Skrypty użytkownika, w podstawowej formie są obiektami bezstanowymi, tzn. nie dostarczają mechanizmu dostępu do poprzednich wartości wejść i wyjść jak i nie umożliwiają przekazywania wartości pomiędzy kolejnymi wywołaniami metod, np.: *gdy użytkownik chce zrealizować funkcje sumującą kolejne wartości pojawiające się na wejściu.*



Aby przekazywać wartości pomiędzy kolejnymi wywołaniami obliczeń konieczne jest wykorzystanie zmiennych globalnych tworzonych przez użytkownika. Takie zmienne tworzone są niezależnie dla każdego skryptu, nie ma problemu z konfliktem nazw zmiennych wykorzystywanych w różnych skryptach (patrz rys. 2) – zmienna `FSum` zdefiniowana jest zarówno w skrypcie „`manual_example_sum`” jak i „`manual_example_integrator`”).

Współdzielenie pamięci, tworzenie uniwersalnych skryptów

Pojedynczy skrypt stworzony przez użytkownika może zostać wykorzystany w kilku blokach funkcyjnych. W takim przypadku, każdy egzemplarz skryptu uruchamiany jest w tym samym zakresie pamięci, tzn. że w przypadku utworzenia zmiennych globalnych z każdego egzemplarza skryptu odwołujemy się do tej samej zmiennej.



Rys. 2). Tworzenie zmiennych użytkownika w skryptach, problem współdzielenia pamięci.

Przykład: (patrz. rys. 2))

Bloki „Script 2” i „Script 2” wykorzystują skrypt „`manual_example_integrator`”, korzystają więc z tej samej zmiennej `FSum`.

Jeśli każdy z uruchomionych egzemplarzy skryptu ma niezależnie korzystać z utworzonych zmiennych globalnych (zwykle o to chodzi) to należy stworzyć odpowiedni obiekt, w którym dostęp do zmiennej realizowany będzie przez identyfikator, unikalny dla każdego egzemplarza skryptu. Rolę takiego identyfikatora pełni jeden z argumentów wywołania występujący we wszystkich metodach skryptu: `_blok_id`. Przykładowo, do realizacji tego zadania może być wykorzystana mapa z biblioteki STL:

```
//Deklaracja zmiennej
std::map<int, double> FSum;

//Odwołanie do poszczególnych "sum"
//poprzez identyfikator bloku
FSum[_blok_id] += _in_val[0] * _sample_rate;
```



Powyższy sposób tworzenia i obsługi zmiennych globalnych pozwala na tworzenie skryptów, które nadają się do wielokrotnego, równoległego wykorzystania w kilku blokach funkcyjnych jednocześnie. Można, w tym przypadku, mówić o tworzeniu wzorców skryptów użytkownika, np. *wzorzec regulatora PID, który następnie będzie wykorzystywany w kilku obwodach regulacji.*

2.3. Etapy tworzenia i uruchamiania skryptu

Utworzenie i uruchomienie skryptu użytkownika składa się z następujących etapów:

- Utworzenie jednego z bloków funkcyjnych biblioteki. Dostęp do tworzenia i edycji skryptów realizowany jest poprzez konfigurację bloków funkcyjnych z biblioteki C/C++ SCRIPTS (przykładowo blok „**Script C/C++**”, rozdział 3.1). Utworzone skrypty zarządzane są niezależnie przez bibliotekę a nie przez każdy z bloków. Dlatego wszystkie dostępne skrypty widoczne są we wszystkich blokach funkcyjnych biblioteki i przez wszystkie te bloki mogą być wykorzystywane.
- Utworzenie nowego skryptu. W ramach tego etapu należy:
 - ⇒ utworzyć nowy skrypt ( File/New),
 - ⇒ napisać treść skryptu ( Prepare/Edit)
 - ⇒ wykonać jego kompilację do kodu źródłowego ( Prepare/Make).Wykonanie poszczególnych etapów sygnalizowane jest zmianą odpowiednich statusów wyświetlanych na liście dostępnych skryptów (patrz rozdział 3.1.1).
- Przypisanie skryptu do bloku funkcyjnego. Dla nowo utworzonego lub już istniejącego bloku funkcyjnego wykorzystującego skrypty należy wybrać jeden z dostępnych skryptów i wykonać operację przypisania ( File/Assign. Jeśli skrypt jest już skompilowany to będzie mógł być od razu wykorzystany w obliczeniach.

Skrypty nieużywane mogą zostać usunięte poprzez wywołanie polecenia  File/Delete.

2.4. Katalog skryptów, kompilacja

Aby umożliwić kompilację skryptów (utworzenie biblioteki dll wykorzystywanej podczas obliczeń przez moduł PExSim) konieczne jest zainstalowanie środowiska BLOODSHED DEV-C++. Plik instalacyjny „devcpp-4.9.9.2_setup.exe” umieszczony jest w następującym podkatalogu systemu AMANDD: „bin_debug\cscripts”. Środowisko można zainstalować wykorzystując domyślne ustawienia konfiguracyjne programu instalacyjnego – nie ma konieczności zmiany jakichkolwiek ustawień.

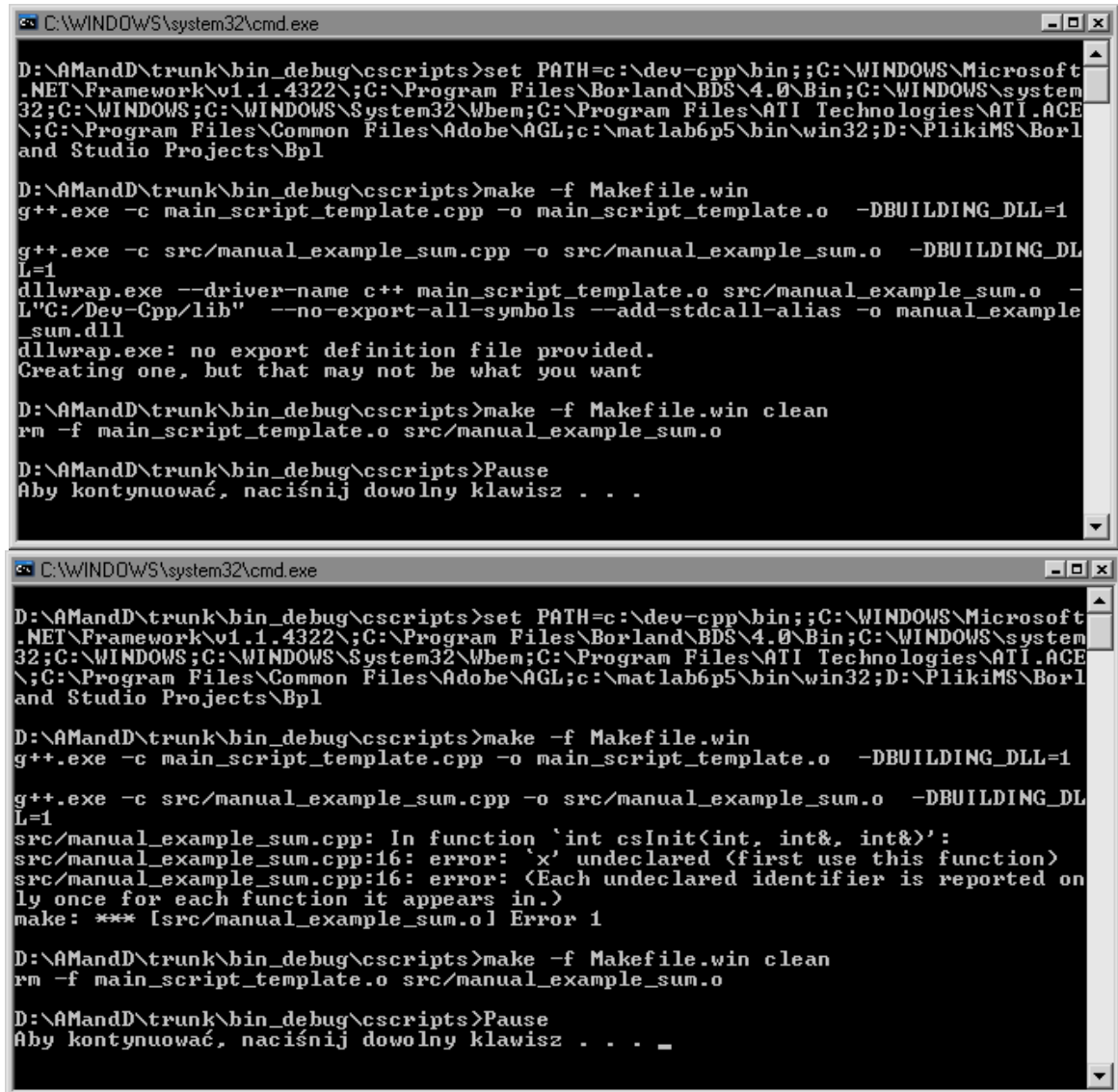
Uwaga!

Środowisko DEV-C++ musi zostać zainstalowane w domyślnym katalogu (C:\Dev-Cpp).



Tworzone skrypty przechowywane są w podkatalogu systemu AMandD: „bin_debug\cscripts\src”. Z tego katalogu należy kopiować pliki w celu dokonania ewentualnej archiwizacji. W katalogu tym znajduje się także wzorzec skryptów „script_template.cpp”. Pliku tego nie wolno kasować ani zmieniać jego nazwy.

Podczas wywołania kompilacji skryptu pojawia się okno tekstowe, w którym wyświetlane są komunikaty kompilatora. W przypadku prawidłowej kompilacji skryptu wyświetlony zostanie odpowiedni komunikat. Użytkownik zostanie także poinformowany o wystąpieniu ewentualnych błędów kompilacji i linkowania (patrz rys. 3))



```
D:\AMandD\trunk\bin_debug\cscripts>set PATH=c:\dev-cpp\bin;;C:\WINDOWS\Microsoft
.NET\Framework\v1.1.4322\;C:\Program Files\Borland\BDS\4.0\Bin;C:\WINDOWS\system
32;C:\WINDOWS;C:\WINDOWS\System32\Wbem;C:\Program Files\ATI Technologies\ATI.ACE
\;C:\Program Files\Common Files\Adobe\AGL;c:\matlab6p5\bin\win32;D:\PlikiMS\Borl
and Studio Projects\Bpl

D:\AMandD\trunk\bin_debug\cscripts>make -f Makefile.win
g++.exe -c main_script_template.cpp -o main_script_template.o -DBUILDING_DLL=1

g++.exe -c src/manual_example_sum.cpp -o src/manual_example_sum.o -DBUILDING_DL
L=1
dllwrap.exe --driver-name c++ main_script_template.o src/manual_example_sum.o -
L"C:/Dev-Cpp/lib" --no-export-all-symbols --add-stdcall-alias -o manual_example
_sum.dll
dllwrap.exe: no export definition file provided.
Creating one, but that may not be what you want

D:\AMandD\trunk\bin_debug\cscripts>make -f Makefile.win clean
rm -f main_script_template.o src/manual_example_sum.o

D:\AMandD\trunk\bin_debug\cscripts>Pause
Aby kontynuować, naciśnij dowolny klawisz . . .
```

```
D:\AMandD\trunk\bin_debug\cscripts>set PATH=c:\dev-cpp\bin;;C:\WINDOWS\Microsoft
.NET\Framework\v1.1.4322\;C:\Program Files\Borland\BDS\4.0\Bin;C:\WINDOWS\system
32;C:\WINDOWS;C:\WINDOWS\System32\Wbem;C:\Program Files\ATI Technologies\ATI.ACE
\;C:\Program Files\Common Files\Adobe\AGL;c:\matlab6p5\bin\win32;D:\PlikiMS\Borl
and Studio Projects\Bpl

D:\AMandD\trunk\bin_debug\cscripts>make -f Makefile.win
g++.exe -c main_script_template.cpp -o main_script_template.o -DBUILDING_DLL=1

g++.exe -c src/manual_example_sum.cpp -o src/manual_example_sum.o -DBUILDING_DL
L=1
src/manual_example_sum.cpp: In function 'int csInit(int, int&, int&)':
src/manual_example_sum.cpp:16: error: 'x' undeclared (first use this function)
src/manual_example_sum.cpp:16: error: (Each undeclared identifier is reported on
ly once for each function it appears in.)
make: *** [src/manual_example_sum.o] Error 1

D:\AMandD\trunk\bin_debug\cscripts>make -f Makefile.win clean
rm -f main_script_template.o src/manual_example_sum.o

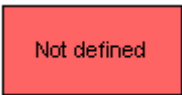
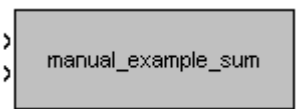
D:\AMandD\trunk\bin_debug\cscripts>Pause
Aby kontynuować, naciśnij dowolny klawisz . . .
```

Rys. 3). Okno informacyjne środowiska kompilacji: przykład prawidłowej (góra) i nieprawidłowej (dół) kompilacji.



3. Opis bloków funkcyjnych

3.1. Script C/C++

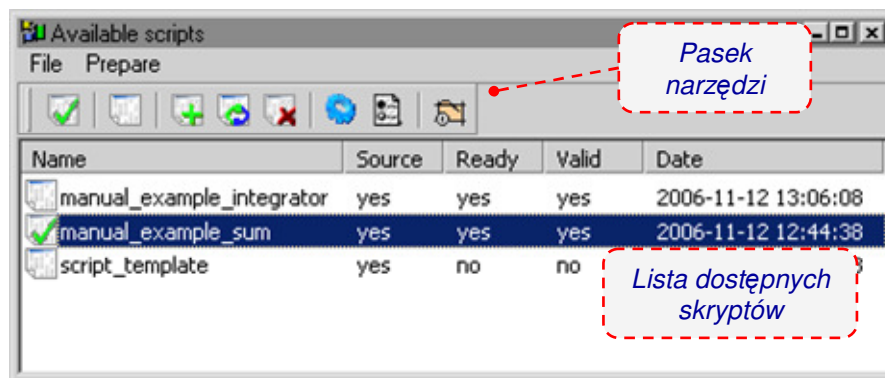
Script C/C++			
Opis		Symbol	
Wykonywanie skryptu użytkownika			Brak dołączonego skryptu
			Dołączony skrypt o podanej nazwie
Funkcja	dowolna, zależna od realizowanego skryptu		
Parametr	Domyślnie	Opis	Typ, zakres
-	-	-	-
Wejścia	Opis		Typ, zakres
$x_1 \dots x_n$	sygnały wejściowy, liczba zależna od realizowanego skryptu		float
Wyjścia	Opis		Typ, zakres
$y_1 \dots y_n$	sygnały wyjściowe, liczba zależna od realizowanego skryptu		float
Błąd	Opis		
-	dowolne, zależne od realizowanego skryptu		
Okno dialogowe		Opis	
Wybór i zarządzania skryptami		Okno uruchamiane jest podwójnym kliknięciem na bloku.	

3.1.1. Okno: wybór i zarządzanie skryptami

Głównym elementem okna wyboru i zarządzania skryptami jest lista, na której umieszczone są wszystkie dostępne skrypty (tworzone przez użytkownika) oraz wzorzec skryptu „script_template”. Skrypt aktualnie przypisany do danego bloku oznaczony jest znakiem . W przypadku, gdy żaden skrypt nie został jeszcze przypisany do bloku, to zaznaczony jest wzorzec skryptu „script_template”. Wzorzec służy jedynie do tworzenia, przez kopię, nowych skryptów. Nie może on być poddawany edycji ani kompilowany. Poszczególne pozycje listy to:

- **Name** Nazwa skryptu, taka jak nazwa pliku skryptu zapisanego na dysku.
- **Source** Zmienna logiczna określająca czy plik źródłowy istnieje.
- **Ready** Zmienna logiczna określająca czy dostępna jest skompilowana wersja skryptu ([do wyjaśnienia](#)).
- **Valid** Zmienna logiczna określająca czy dostępna jest skompilowana wersja skryptu.
- **Date** Czas i data ostatniej modyfikacji kodu źródłowego skryptu ([do wyjaśnienia](#)).

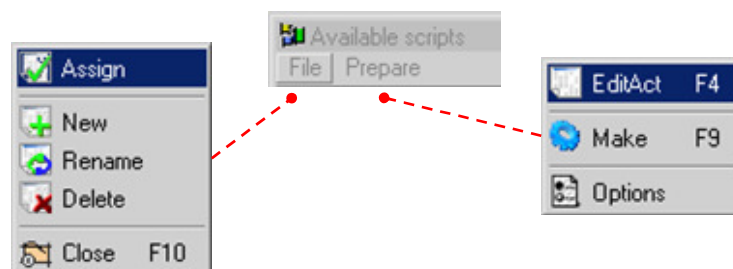




Rys. 4). Okno konfiguracyjne wyboru i zarządzania skryptami.

Wszystkie polecenia związane z zarządzaniem skryptami dostępne są poprzez menu. Wybrane, najczęściej używane polecenia udostępnione zostały także na pasku narzędzi. Poszczególne elementy menu pokazano na rys. 5). Wyróżniono następujące grupy:

- *File* Polecenia zarządzania skryptami.
- *Prepare.* Polecenia związane z przygotowaniem skryptu do uruchomienia.



Rys. 5). Elementy menu okno konfiguracyjne wyboru i zarządzania skryptami.

Operacje na plikach

Użytkownik ma dostęp do następujących funkcji zarządzania skryptami:

-  **File/Assign** Przypisanie do bloku wybranego na liście skryptu. Wzorzec skryptu `script_template` nie może zostać przypisany.
-  **File/New** Utworzenie nowego skryptu na podstawie wzorca „`script_template`”.
-  **File/Rename** Zmiana nazwy wybranego na liście skryptu. Nazwa wzorca skryptu `script_template` nie może zostać zmieniona.
-  **File/Delete** Usunięcie wybranego na liście skryptu. Wzorzec skryptu `script_template` nie może zostać usunięty.
-  **File/Close** Zamknięcie okna konfiguracji.



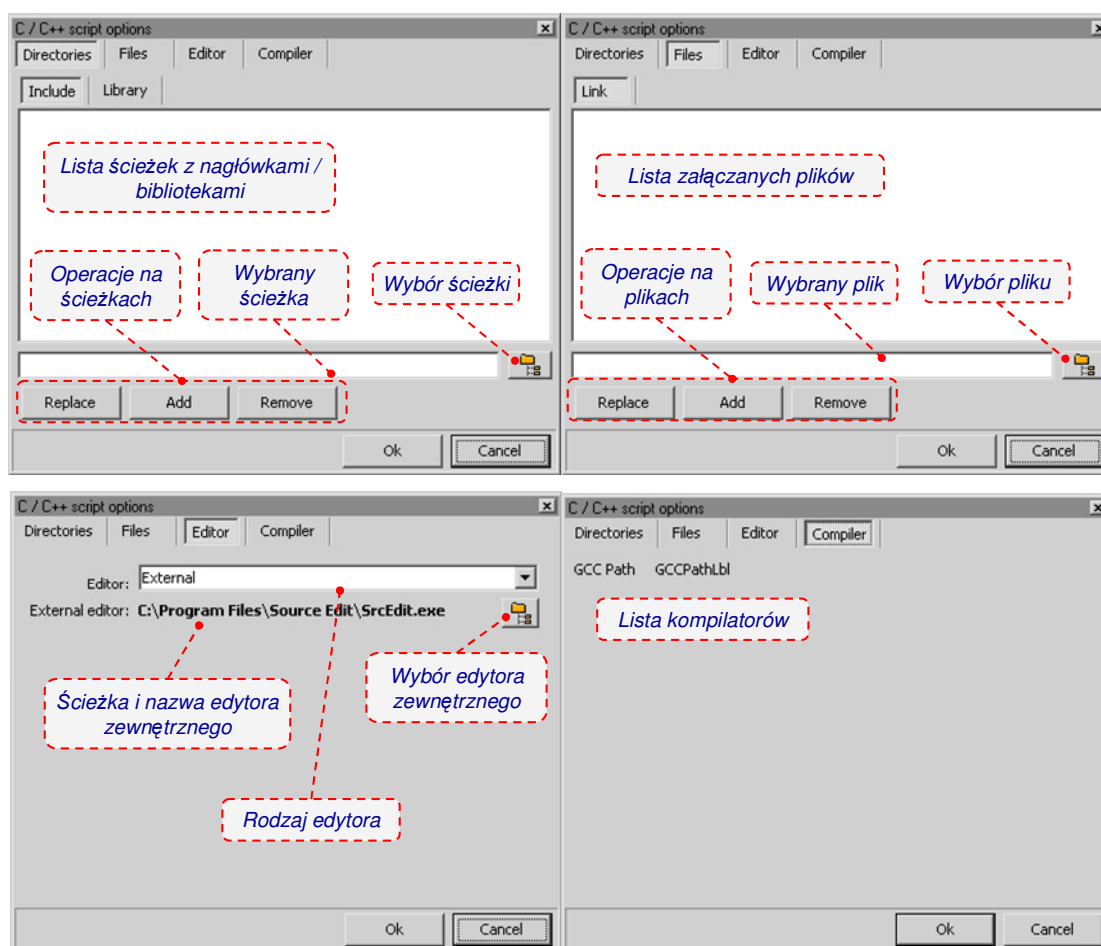
Przygotowanie skryptów

Użytkownik ma dostęp do następujących funkcji związanych z przygotowaniem skryptu do uruchomienia:

- | | | |
|---|-----------------|--|
|  | Prepare/Edit | Przejsie do edycji wybranego na liście skryptu. Wzorzec skryptu <code>script_template</code> nie może być edytowany. |
|  | Prepare/Make | Kompilacja wybranego na liście skryptu. Wzorzec skryptu <code>script_template</code> nie może być kompilowany. |
|  | Prepare/Options | Wyświetlenie okna służącego do ustawienia opcji konfiguracji środowiska edycji skryptów. Zostały one omówione w rozdziale Błąd! Nie można odnaleźć źródła odwołania.. |

3.1.2. Opcje konfiguracji środowiska edycji skryptów

Dostęp do konfiguracji środowiska edycji skryptów realizowany jest przez okno opcji uruchamiane poleceniem  *Prepare/Options*. Okno pokazano na rys. 6).



Rys. 6). Okno opcji konfiguracji środowiska edycji skryptów.



Parametry konfiguracyjne podzielono na cztery grupy:

- **Directories** Lista katalogów, które są przeszukiwane w celu dołączenia wskazanych plików nagłówkowych (grupa *Include*) i bibliotek (grupa *Library*). Użytkownik ma możliwość dodania nowej ścieżki oraz zamiany lub usunięcia już istniejącej. ([do wyjaśnienia](#))
- **Files** Lista plików, które mogą być załączane podczas kompilacji skryptów. Użytkownik ma możliwość dodania nowego pliku oraz zamiany lub usunięcia już istniejącego. ([do wyjaśnienia](#))
- **Editor** Wybór środowiska wykorzystywanego do edycji skryptów.
- **Compiler** Opcje wyboru kompilatora. (w aktualnej wersji wyświetlany jest jedynie nazwa domyślnego konfiguratora).

Działanie komend: *Replace*, *Add*, *Remove*, *Ok* i *Cancel* jest standardowe, zgodne z konwencją przyjętą w systemie Windows. Szczegółowy opis opcji przedstawiono w tabeli 1.

Tabela 1. Opcje konfiguracji i edycji skryptów.

Opcja	Grupa	Wartość		Opis
		Zakres	Domyślnie	
<i>Editor</i>	<i>Editor</i> (opcje edytora skryptów)	<i>{Internal, External}</i>	<i>Internal</i>	Pole wyboru określające czy do edycji skryptów ma być wykorzystany edytor wewnętrzny (<i>Internal</i>) czy zewnętrznym, wskazany przez użytkownika (<i>External</i>).
<i>External editor</i>		<i>URL</i>	–	Położenie oraz nazwa zewnętrznego programu do edycji, który został wybrany przez użytkownika.



Dodatek A. Kod źródłowy wzorca skryptów C/C++ „script_template.cpp”

```
#include "../main_script_template.h"
#include <stdlib.h>

/** Initiation.

This method is called always when the script is assigned to the object.
@param _block_id Unique ID of parent block - it is impossible to have
                two block with same ID (managed automatically)
@param _input_count A place for inputs number for block (returning value)
@param _output_count A place for nubmer of outputs (returning value).
@return 0 if successful, error code to be displayed otherwise */
extern "C" DLLIMPORT int csInit(      const int _block_id,
                                     int& _input_count,
                                     int& _output_count)

{
    _input_count = 3;
    _output_count = 2;
    return 0;
}

/** Main calculate routine (needs to be implemented)

Here you can calculate and set output values, and set their statuses.

@param _block_id Unique ID of parent block - it is impossible to have
                two block with same ID (managed automatically)
@param _sample_rate Sample time (in seconds)
@param _in_val input values table
@param _in_stat input status table
@param _in_size size of input tables
@param _out_val input values table (returning value)
@param _out_stat input status table (returning value)
Recognised statuses are:
<ul>
<li> 0 - OK
<li> 1 - Temporary
<li> 2 - Bad
<li> 4 - Uncertain
<li> 8 - High limit
<li> 16 - Low limit
<li> 32 - OVR
<li> 64 - Missing
<li> 128 - Not connected
<li> 256 - Not ready
<li> 0x08000000 - Unknown
</ul>
Same status codes are used in inputs table. Statuses can be combined
using | operator, eg to set temporary and uncertain, use following
code: int stat = 1 | 4;
to check if bad is set, use
if (stat & 2) { ... }
@param _out_size size of input tables
@return 0 if successful, error code to be displayed otherwise */
extern "C" DLLIMPORT int csCalculate(  const int _block_id,
                                       const double _sample_rate,
                                       const double* _in_val,
                                       const int* _in_status,
                                       const int _in_size,
                                       double* _out_val,
                                       int* _out_status,
                                       const int _out_size)

{
    double v = 0;
    for (int i = 0; i < _in_size; i++) v += _in_val[i];
    v *= _sample_rate;
    for (int i = 0; i < _out_size; i++)
    {
        _out_val[i] = v;
        _out_status[i] = _in_status[i];
    }
    return 0;
}
```

```

/*****
 * Next functions can be implemented, but in normal way there is no need
 * to do that
 *****/

/** Simulation initiation

This method is called always directly before simulation starts.

@param _block_id Unique ID of parent block - it is impossible to have
                  two block with same ID (managed automatically)
@param _sample_rate Sample time (in seconds)
@return 0 if successful, error code to be displayed otherwise */
extern "C" DLLIMPORT int csBeforeSimStart( const int _block_id,
                                           const double _sample_rate)
{
    return 0;
}

/** Before simulation step

This method is called always directly before simulation step will be done. Inputs
do not have yet proper values!

@param _block_id Unique ID of parent block - it is impossible to have
                  two block with same ID (managed automatically)
@param _sample_rate Sample time (in seconds)
@param _in_val input values table
@param _in_stat input status table
@param _in_size size of input tables
@return 0 if successful, error code to be displayed otherwise */
extern "C" DLLIMPORT int csBeforeCalc( const int _block_id,
                                       const double _sample_rate,
                                       const double* _in_val,
                                       const int* _in_status,
                                       const int _in_size)
{
    return 0;
}

/** After simulation step

This method is called always directly after simulation step was done. Inputs
have now final values.

@param _block_id Unique ID of parent block - it is impossible to have
                  two block with same ID (managed automatically)
@param _sample_rate Sample time (in seconds)
@param _in_val input values table
@param _in_stat input status table
@param _in_size size of input tables
@return 0 if successful, error code to be displayed otherwise */
extern "C" DLLIMPORT int csAfterCalc( const int _block_id,
                                       const double _sample_rate,
                                       const double* _in_val,
                                       const int* _in_status,
                                       const int _in_size)
{
    return 0;
}

/** Routine called when simulation was stopped

@param _block_id Unique ID of parent block - it is impossible to have
                  two block with same ID (managed automatically)
@param _sample_rate Sample time (in seconds)
@return 0 if successful, error code to be displayed otherwise */
extern "C" DLLIMPORT int csAfterSimStop( const int _block_id,
                                          const double _sample_rate)
{
    return 0;
}

```

